QueryAPIGate HOW-TO GUIDE 47 · AGENTS

Tell AI agents what your data means, and see what they did

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=47-tell-agents-what-your-data-means-and-see-what-they-did

Time: 15 minutes. **You'll end up with:** an agent that is told what each table and column means - imported from your database's own comments, or written in the Console - and the joins and metric definitions to use, so it writes the right SQL; and, afterwards, a view of what every agent asked, read and was refused, with an alert when one suddenly reads far more than it usually does.

> **Stable** - descriptions and agent usage are under the compatibility promise since 0.30 (what that means).

Every command and response below was run against a real server and its MCP server, with the example APIs loaded, a PostgreSQL 16 database as a second connection, and a real MCP client (the Python SDK) as the agent. `demo-key` is an admin; the agent is a scoped key, `rentals-bot`.

Step 1: Import what your database already says

Many databases already carry comments. In PostgreSQL:

```sql
COMMENT ON TABLE orders IS 'One row per order, from checkout';
COMMENT ON COLUMN orders.amt IS 'Net revenue in EUR, refunds excluded';
COMMENT ON COLUMN orders.status IS 'open, paid or refunded';
```

Bring them in:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections/shop/descriptions/import -H 'X-API-Key: demo-key'
```

```json
{"found": 3, "imported": 3, "kept": 0, "items": [
  {"table": "orders", "column": null, "description": "One row per order, from checkout", "source":
"database", ...},
  {"table": "orders", "column": "amt", "description": "Net revenue in EUR, refunds excluded", "source":
"database", ...},
  {"table": "orders", "column": "status", "description": "open, paid or refunded", "source": "database",
...}]}
```

PostgreSQL, MySQL, ClickHouse and DuckDB comments can be imported. Run it again after the comments change: a description someone wrote in QueryAPIGate (`"source": "written"`) is kept, unless you send `{"overwrite": true}`.

Read an import before relying on it. Whoever can `COMMENT` in the database writes what agents will be told.

Step 2: Write the rest - and the connection's notes

What the database doesn't say, write. The connection's own **notes** (`"table": null`) are where joins and metric definitions go:

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/connections/examples/descriptions -H 'X-API-Key: demo-key' \
  -H 'Content-Type: application/json' -d '{"items": [
  {"table": "rental", "column": null, "description": "One row per film rented: who, which film, when"},
  {"table": "rental", "column": "return_date", "description": "When it came back; null while still
out"},
  {"table": "rental", "column": "amount", "description": "Rental fee in EUR, before late fees"},
  {"table": "customer", "column": "email", "description": "Where receipts go"},
  {"table": "customer", "column": "country", "description": "ISO 3166 two-letter code"},
  {"table": null, "description": "Join rental to customer on customer_id, and to film on film_id. Late
fees are in payment, not rental: revenue is SUM(payment.amount)."}
]}'
```

A `null` or empty description removes one. In the Console: **API Designer → Schema**, ✎ beside a table for it and its columns, and ✎ **Describe connection** for the notes and **Import comments**. Owners and admins may write them; every change is in the audit log with what it said before.

Step 3: What the agent is told

The agent's key reaches three tables, with the email hashed:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "rentals-bot", "connections": ["examples"], "allowed_tables": ["rental", "customer", "film"],
  "mask": [{"column": "customer.email", "action": "hash"}]
}'
```

Its `list_tables` now carries the descriptions - and says what it gets masked:

```json
{"name": "email", "type": "TEXT", "description": "Where receipts go", "masked": "hash"}
{"name": "country", "type": "TEXT", "description": "ISO 3166 two-letter code"}
{"name": "return_date", "type": "TEXT", "description": "When it came back; null while still out"}
{"name": "amount", "type": "REAL", "description": "Rental fee in EUR, before late fees"}
```

And the MCP server offers it a **data dictionary**, a resource it can read in one go before writing SQL - `resources/list` gives `queryapigate://connections/examples/dictionary`:

```markdown
# Data dictionary: examples

## customer

- `customer_id` (INTEGER)
- `first_name` (TEXT)
- `last_name` (TEXT)
- `email` (TEXT) - Where receipts go - masked for you (hash)
- `country` (TEXT) - ISO 3166 two-letter code

## film
...

## rental
One row per film rented: who, which film, when

- `rental_id` (INTEGER)
...
- `return_date` (TEXT) - When it came back; null while still out
- `amount` (REAL) - Rental fee in EUR, before late fees
```

Two things are missing, on purpose: the `payment` table, which the key can't reach - and so the connection's notes, which mention it. A caller restricted to some tables never gets the notes; give an agent that needs them the whole connection, or put what it needs in its tables' descriptions.

Step 4: Describe a saved query's columns

A saved query's own result columns are described in its definition - `columns`:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "late_returns", "connection_name": "examples", "description": "Rentals not returned yet",
"publish": true,
  "sql": "SELECT rental_id, customer_id, rental_date FROM rental WHERE return_date IS NULL ORDER BY
rental_date",
  "columns": {"rental_id": "The rental, as in rental.rental_id", "rental_date": "When it went out, in
UTC"}
}'
```

Its MCP tool's output schema carries them:

```json
{"type": "array", "items": {"type": "object", "properties": {
  "rental_id": {"description": "The rental, as in rental.rental_id"},
  "rental_date": {"description": "When it went out, in UTC"}}}}
```

Step 5: See what the agent did

After the agent ran `late_returns`, read customers twice, read rentals, and tried the `payment` table it can't reach:

```shell
curl 'http://127.0.0.1:5000/api/v1/usage/agents?hours=1' -H 'X-API-Key: demo-key'
```

```json
{"since": "2026-10-11 09:29:15", "until": "2026-10-11 10:29:16", "sample_rate": 1.0, "truncated": false,
"items": [
  {"agent": "rentals-bot", "calls": 5, "rows": 800, "errors": 1, "masked_calls": 2,
   "first_seen": "2026-10-11 10:29:13", "last_seen": "2026-10-11 10:29:13", "connections": ["examples"],
   "queries": {"late_returns": 1}, "adhoc_calls": 4, "refusals": {"table_not_allowed": 1},
   "top_statements": [
     {"sql": "SELECT customer_id, email, country FROM customer", "count": 2, "rows": 400, "errors": 0,
...},
     {"sql": "SELECT rental_id, amount FROM rental", "count": 1, "rows": 200, "errors": 0, ...},
     {"sql": "SELECT sum(amount) FROM payment", "count": 1, "rows": 0, "errors": 1, ...}]}]}
```

- **`refusals`** is what to read first: `table_not_allowed`, `mask_unresolvable` or `read_only` are an agent trying what its grant doesn't allow. Here it went for revenue in `payment` - the notes it couldn't see would have told it that's where revenue is. Grant the table, or say what to use in the tables it has.
- **`masked_calls`**: the two customer reads, with `email` hashed.
- An agent is an API key, or a signed-in user on the MCP server (guide 46): each person's agents under their own name.

In the Console: **Observability** → **Agents**, busiest first, a row opening to the agent's queries and statements.

Step 6: Be told when one reads far more than usual

`agent_unusual_volume` fires when an agent reads at least `QUERYAPIGATE_ALERT_AGENT_ROWS` rows (default 10,000) in an hour, and ten times its own average hour. With the setting at 100 for this run, the new agent's 800 rows did it:

```json
{"id": "agent_unusual_volume:rentals-bot", "severity": "warning", "kind": "agent_unusual_volume",
 "title": "Agent 'rentals-bot' is reading far more than usual",
 "detail": "800 rows over MCP in the last hour, against next to nothing before. Check what it is asking
on the Agents screen; narrow its grant, or lower its rate_limit, if it shouldn't.",
 "target": {"type": "agent", "name": "rentals-bot"}, ...}
```

It's on the Alerts screen's **Agents** tab, and in `GET /api/v1/alerts` for your own monitoring. A loop gone wrong and data being pulled out look the same here - the Agents screen tells them apart.

Good to know

- **Names match case-insensitively**, and descriptions are kept for the connection's own database: browsing another one on the same server (`?database=`) shows none.
- **Agent usage comes from run history**, so it holds what history holds: calls refused before they ran (no credential, a rate limit) are only in `/metrics`; with `QUERYAPIGATE_HISTORY_SAMPLE_RATE` below 1, successful runs are a sample (`sample_rate` says so); with `QUERYAPIGATE_HISTORY_ADHOC_SQL=hash`, statements are grouped by hash.
- **Every administrator role can read agent usage**, auditors included - it shows the SQL agents ran, as run history does.

Next steps

- Hide personal data from a partner or an AI agent - the masking behind `"masked": "hash"`.
- Let AI clients sign in through your identity provider.
- API reference: descriptions and agent usage.

READ NEXT

Sign in to the Console through your identity provider

QUESTIONS OR PROBLEMS

Discord · GitHub issues

LATEST VERSION OF THIS PAGE

anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=47-tell-agents-what-your-data-means-and-see-what-they-did

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit

@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate